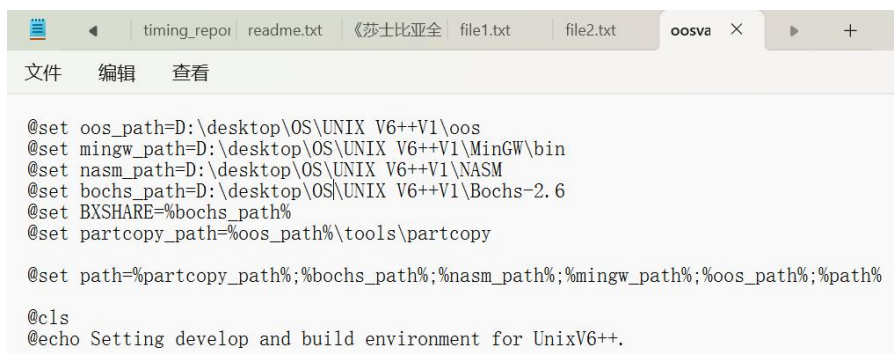


实验一： Unix V6++安装配置

Part1: 安装 Unix V6++系统

1.首先编辑 oosvars_mingw.bat 文件，指向我的文件路径

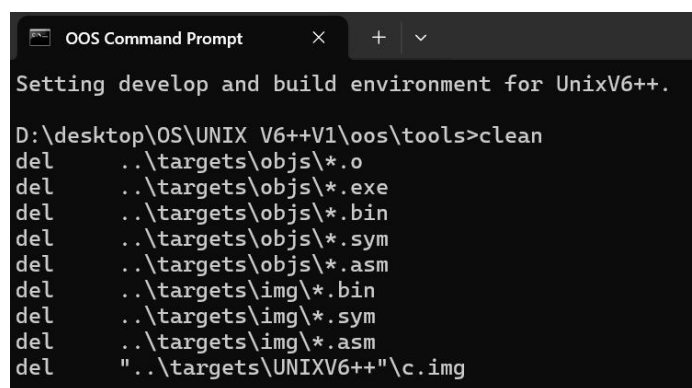


```
@set oos_path=D:\desktop\OS\UNIX V6++V1\oos
@set mingw_path=D:\desktop\OS\UNIX V6++V1\MinGW\bin
@set nasm_path=D:\desktop\OS\UNIX V6++V1\NASM
@set bochs_path=D:\desktop\OS\UNIX V6++V1\Bochs-2.6
@set BXSHARE=%bochs_path%
@set partcopy_path=%oos_path%\tools\partcopy

@set path=%partcopy_path%;%bochs_path%;%nasm_path%;%mingw_path%;%oos_path%;%path%

@cls
@echo Setting develop and build environment for UnixV6++.
```

2.执行批处理命令 clean，清除已有全部目标文件（二进制文件，.o），擦除已有镜像文件 c.img

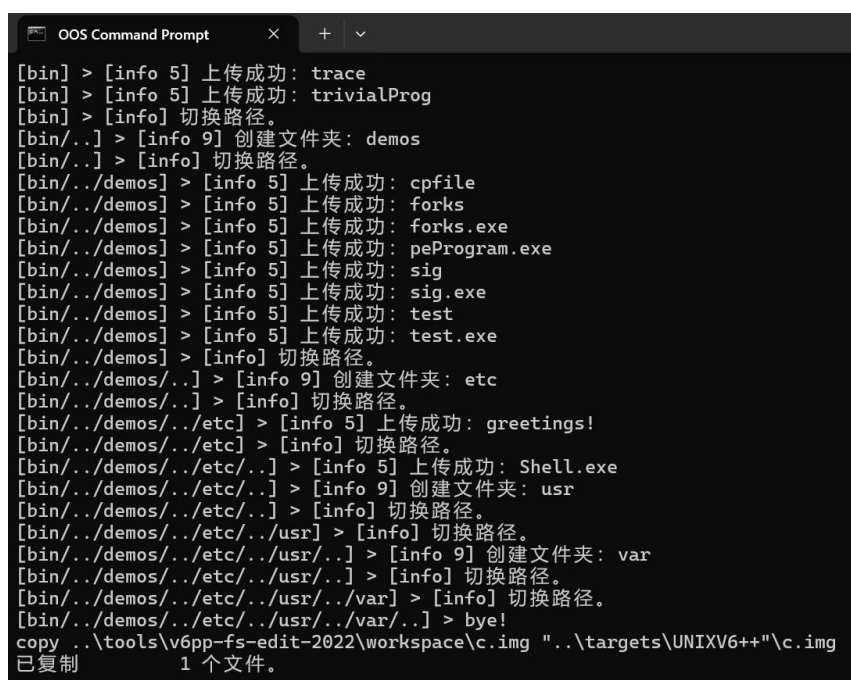


```
OOS Command Prompt

Setting develop and build environment for UnixV6++.

D:\desktop\OS\UNIX V6++V1\oos\tools>clean
del    ..\targets\objs\*.o
del    ..\targets\objs\*.exe
del    ..\targets\objs\*.bin
del    ..\targets\objs\*.sym
del    ..\targets\objs\*.asm
del    ..\targets\img\*.bin
del    ..\targets\img\*.sym
del    ..\targets\img\*.asm
del    "..\targets\UNIXV6++"c.img
```

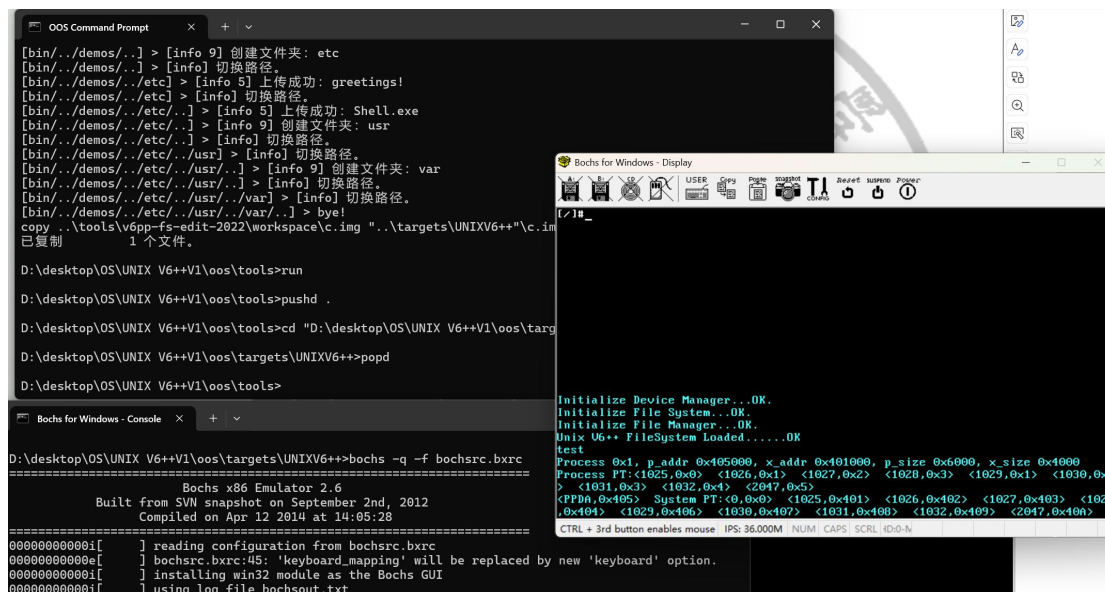
3.执行批处理命令 all，重新编译内核和应用程序、制作新的系统镜像文件 c.img



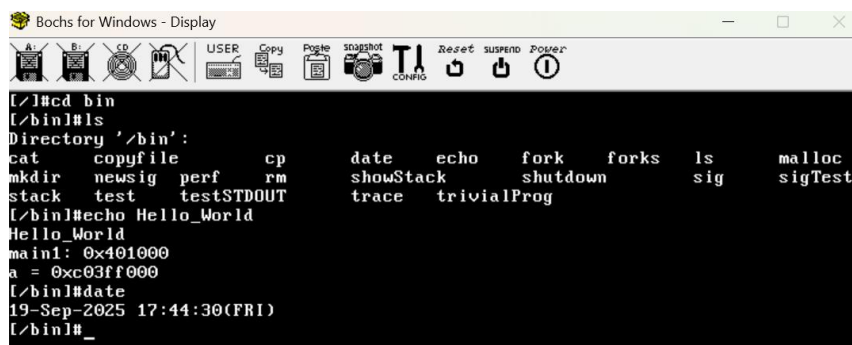
```
OOS Command Prompt

[bin] > [info 5] 上传成功: trace
[bin] > [info 5] 上传成功: trivialProg
[bin] > [info] 切换路径。
[bin/..] > [info 9] 创建文件夹: demos
[bin/..] > [info] 切换路径。
[bin/./demos] > [info 5] 上传成功: cpfile
[bin/./demos] > [info 5] 上传成功: forks
[bin/./demos] > [info 5] 上传成功: forks.exe
[bin/./demos] > [info 5] 上传成功: peProgram.exe
[bin/./demos] > [info 5] 上传成功: sig
[bin/./demos] > [info 5] 上传成功: sig.exe
[bin/./demos] > [info 5] 上传成功: test
[bin/./demos] > [info 5] 上传成功: test.exe
[bin/./demos] > [info] 切换路径。
[bin/./demos/..] > [info 9] 创建文件夹: etc
[bin/./demos/..] > [info] 切换路径。
[bin/./demos/./etc] > [info 5] 上传成功: greetings!
[bin/./demos/./etc] > [info] 切换路径。
[bin/./demos/./etc/..] > [info 5] 上传成功: Shell.exe
[bin/./demos/./etc/..] > [info 9] 创建文件夹: usr
[bin/./demos/./etc/..] > [info] 切换路径。
[bin/./demos/./etc/./usr] > [info] 切换路径。
[bin/./demos/./etc/./usr/..] > [info 9] 创建文件夹: var
[bin/./demos/./etc/./usr/..] > [info] 切换路径。
[bin/./demos/./etc/./usr/./var] > [info] 切换路径。
[bin/./demos/./etc/./usr/./var/..] > bye!
copy ..\tools\v6pp-fs-edit-2022\workspace\c.img "..\targets\UNIXV6++"c.img
已复制      1 个文件。
```

4.运行 UNIX V6++系统



5. 执行几条简单的 Unix 命令



注：这里的 `echo` 函数和实验指导中的相比，多显示了两行，我猜测是 `main` 函数代码和局部变量 `a` 在虚拟内存的存放地址。随后查看 `echo` 函数的源代码，发现 `main1: 0x401000` 这一行是 `echo` 程序的 `main` 函数代码的地址，`a = 0xc03ff000` 则是一个设置好的地址。这个地址位置很高很高，所以我猜测可能是在说明变量的栈地址。

```
int main1(int argc, char* argv[])
{
    int i;
    char line[100];
    for( i = 1; i < argc; i++)
    {
        printf("%s ", argv[i]);
    }

    printf("\n");
    printf("main1: 0x%x\n",&main1);
    unsigned long a = (unsigned long)0xc03ff000;
    printf("a = 0x%x\n", (unsigned long)a);
    exit(0);
}
```

Part2: 配置 UNIX V6++的集成开发环境

2.1 给出每一个 oos/src/下的子目录中所包含文件的用途

1.boot: 加电引导汇编程序

boot 存放计算机启动最初阶段的代码，也是操作系统最先执行的一步。当计算机通电时，这一块程序负责将磁盘 0 号扇区的引导加载代码复制到内存中并执行，用于初始化硬件并为内核的运行做准备。

2.dev: 设备驱动

dev 存放与硬件设备相关的驱动程序代码。操作系统通过这些代码来控制具体的硬件，比如硬盘、键盘等。比如，DMA.cpp 实现直接内存访问（DMA）相关的逻辑与封装；BufferManager.cpp 实现块缓存/缓冲区管理等等。

3.fs: 文件系统

fs 存放文件系统实现的代码。创建文件、读写数据、建立目录等所有操作的实现代码都在这里。它负责管理数据在硬盘上的组织结构，并向上层提供统一的文件操作接口。

4.include: 头文件

include 包含整个内核项目的所有公共接口和类型定义。比如 Allocator.h, KernelAllocator.h 是内存分配相关接口；SystemCall.h, Exception.h 是系统调用与异常处理接口等等。

5.interrupt: 中断处理

interrupt 负责处理硬件中断和软件异常。DiskInterrupt.cpp 是磁盘中断处理程序，响应硬盘控制器发出的中断、唤醒等待的 IO 请求、处理错误和更新状态；Exception.cpp 是 CPU 异常处理逻辑；KeyboardInterrupt.cpp 是键盘中断处理；SystemCall.cpp 是系统调用入口；TimeInterrupt.cpp 是定时、时钟中断处理

6.kernel: 内核核心

存放操作系统的核心代码。比如 main.cpp 应该是内核的入口点，在引导程序之后加载（可能是上课讲的 1 号到 200 号扇区？）；Utility.cpp 是内核内用的各种通用工具函数的代码实现（如内存复制、字符串操作等）。

7.lib: 库函数

lib 应该是存放用户程序或内核可重用的函数集合（如字符串、内存、IO 等）

8.machine:

可能是与 CPU 架构相关的代码？（因为文件命名包含 CMOS、Chip 等，应该是底层硬件相关代码）

9.mm: 内存管理

mm 存放管理物理内存和虚拟内存的代码。包括内存的分配与回收、页表的建立与维护、缺页中断的处理等。

10.pe: PE 格式处理

解析 Windows/PE 格式可执行文件，实现读取 PE 头、节表、导入表及映射到内存的逻辑。

11.proc: 进程管理

比如 Process.cpp 负责进程结构及其操作，比如创建、销毁、状态切换、保存/恢复上下文。ProcessManager.cpp 是调度器/进程表管理。

12.program: 用户级程序

这里存放用户在终端使用的命令，比如 ls、cat 等等的实现代码。

13.shell: 命令行

可能用于对命令进行预处理、解析和执行

14.test: 测试

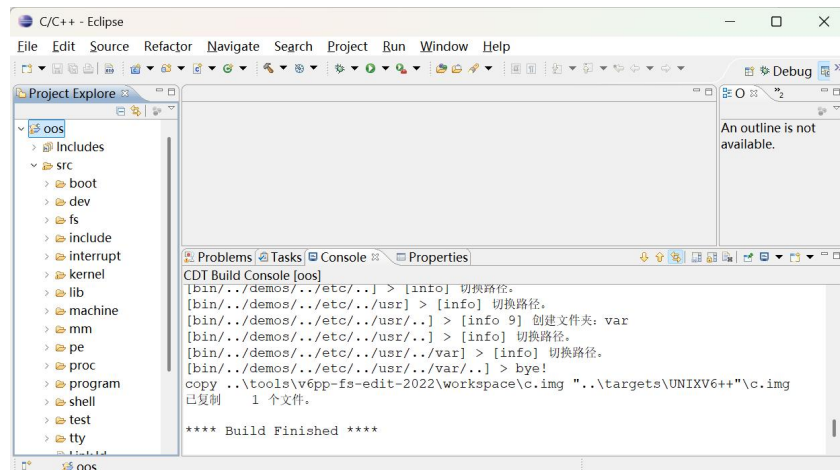
测试操作系统各个模块功能的测试用例

15.tty: 终端设备

负责管理终端比如键盘输入的驱动和逻辑。

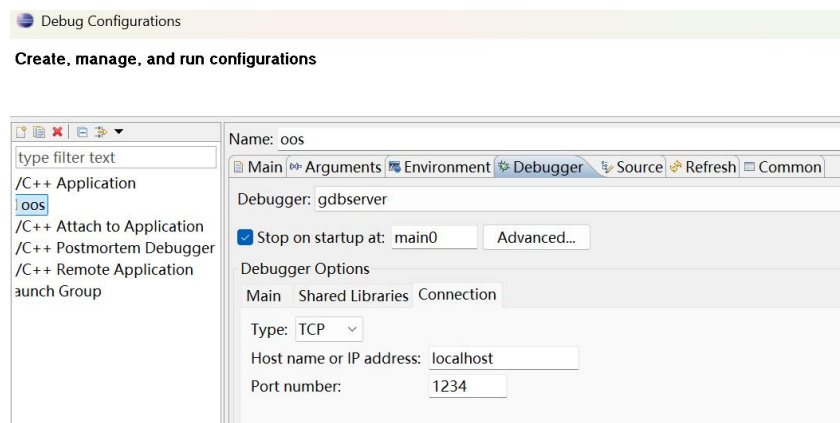
2.2 搭建基于 eclipse 的 gdb 远程调试开发环境

1.首先配置好编译环境



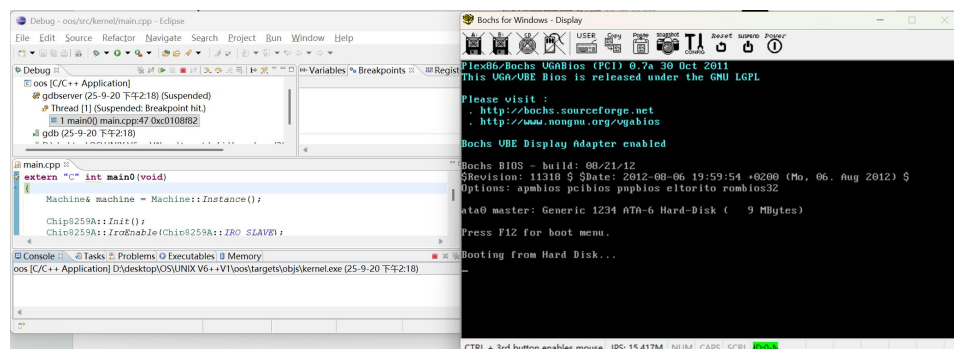
2.为 UNIX V6++配置 gdb 远程调试

```
#####  
# bochsrc.txt file for DLX Linux disk image.  
#####  
gdbstub: enabled=1, port=1234, text_base=0, data_base=0, bss_base=0
```

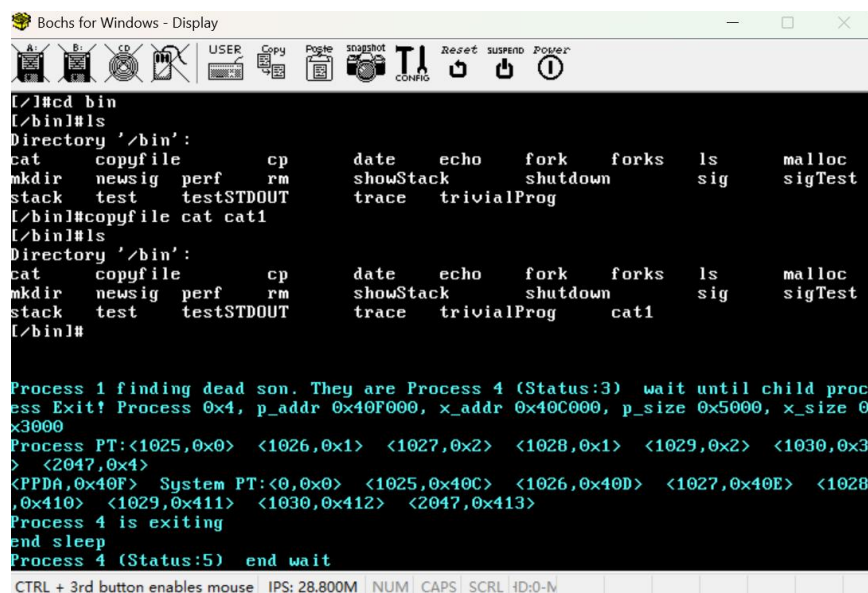


3.调试 UNIX V6++

开始调试



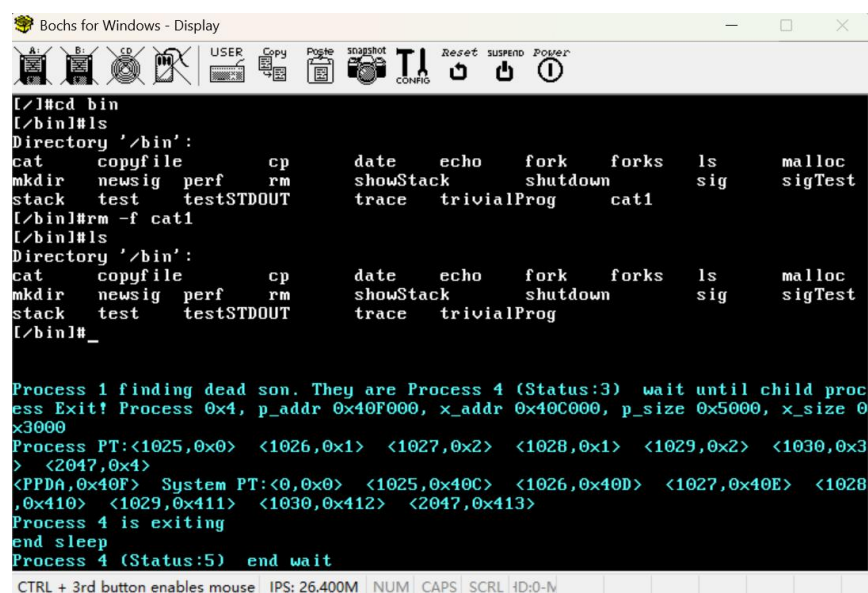
在命令行界面输入 UNIX 命令 ls、copyfile，在 debug 界面点击 resume 按钮



```
Bochs for Windows - Display
[/]#cd bin
[/bin]#ls
Directory '/bin':
cat    copyfile    cp      date    echo    fork    forks    ls      malloc
mkdir  newsig    perf    rm      showStack shutdown sig      sigTest
stack  test      testSTDOUT trace    trivialProg
[/bin]#copyfile cat cat1
[/bin]#ls
Directory '/bin':
cat    copyfile    cp      date    echo    fork    forks    ls      malloc
mkdir  newsig    perf    rm      showStack shutdown sig      sigTest
stack  test      testSTDOUT trace    trivialProg    cat1
[/bin]#

Process 1 finding dead son. They are Process 4 (Status:3) wait until child process Exit! Process 0x4, p_addr 0x40F000, x_addr 0x40C000, p_size 0x5000, x_size 0x3000
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x1> <1029,0x2> <1030,0x3> <2047,0x4>
<PPDA,0x40F> System PT:<0,0x0> <1025,0x40C> <1026,0x40D> <1027,0x40E> <1028,0x410> <1029,0x411> <1030,0x412> <2047,0x413>
Process 4 is exiting
end sleep
Process 4 (Status:5) end wait
CTRL + 3rd button enables mouse | IPS: 28.800M | NUM | CAPS | SCRL | ID:0-N
```

退出 debug 模式再跑一遍，发现复制的文件 cat1 还在



```
Bochs for Windows - Display
[/]#cd bin
[/bin]#ls
Directory '/bin':
cat    copyfile    cp      date    echo    fork    forks    ls      malloc
mkdir  newsig    perf    rm      showStack shutdown sig      sigTest
stack  test      testSTDOUT trace    trivialProg    cat1
[/bin]#rm -f cat1
[/bin]#ls
Directory '/bin':
cat    copyfile    cp      date    echo    fork    forks    ls      malloc
mkdir  newsig    perf    rm      showStack shutdown sig      sigTest
stack  test      testSTDOUT trace    trivialProg
[/bin]#

Process 1 finding dead son. They are Process 4 (Status:3) wait until child process Exit! Process 0x4, p_addr 0x40F000, x_addr 0x40C000, p_size 0x5000, x_size 0x3000
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x1> <1029,0x2> <1030,0x3> <2047,0x4>
<PPDA,0x40F> System PT:<0,0x0> <1025,0x40C> <1026,0x40D> <1027,0x40E> <1028,0x410> <1029,0x411> <1030,0x412> <2047,0x413>
Process 4 is exiting
end sleep
Process 4 (Status:5) end wait
CTRL + 3rd button enables mouse | IPS: 26.400M | NUM | CAPS | SCRL | ID:0-N
```

3.3 通过调试环境的配置和运行，查阅相关资料，谈谈对远程调试的理解

我觉得远程调试就是一种将被调试的程序与运行调试器的环境分离开来的调试方法，两者通过一个通信链路进行交互。被调试的程序运行环境就是运行 UNIX V6++的 bochs 虚拟机，而调试器的运行环境是 Windows 桌面系统，上面运行着 Eclipse 调试器，用来编写代码、设置断点、查看变量等调试操作。

之所以叫远程调试，是因为 Eclipse 并没有直接运行在 Bochs 虚拟机里，而是通过一种通信机制，远程地“遥控”和“监视”着 Bochs 里内核的运行状态。

为什么一定需要远程调试呢？因为调试器本身也需要运行在一个操作系统上。如果直接在 UNIXV6++上运行 Eclipse，当内核因为断点而暂停时，整个操作系统都停止运行了，调试器自己也无法工作。因此，必须将调试器放在一个独立的操作系统（主机）上，去调试另一个系统（目标机）。

当我启动了 debug 模式后，远程调试整个流程是这样的：首先，在 Eclipse 里点击“单步执行”，然后 Eclipse 将该操作转换成 GDB 远程协议命令，并通过本地 TCP/IP 连接发送给 Bochs。Bochs 的 gdbstub 接收命令，控制虚拟机 CPU 执行一条指令并暂停。gdbstub 将执行后的 CPU 状态(如寄存器值)打包成协议数据，通过 TCP/IP 连接返回给 Eclipse。最后 Eclipse 解析数据并更新界面中的变量和内存视图。

part3: 编写一个新的应用程序 helloworld

1.编写 helloworld.c 程序

```
C helloworld.c X C fork.c
src > program > C helloworld.c > main1(int, char * [])
1  #include <stdio.h>
2
3  int main1(int argc, char *argv[])
4  {
5      printf("Welcome to UnixV6++!\n");
6      return 0;
7  }
```

2.在 makefile 中添加构建目标

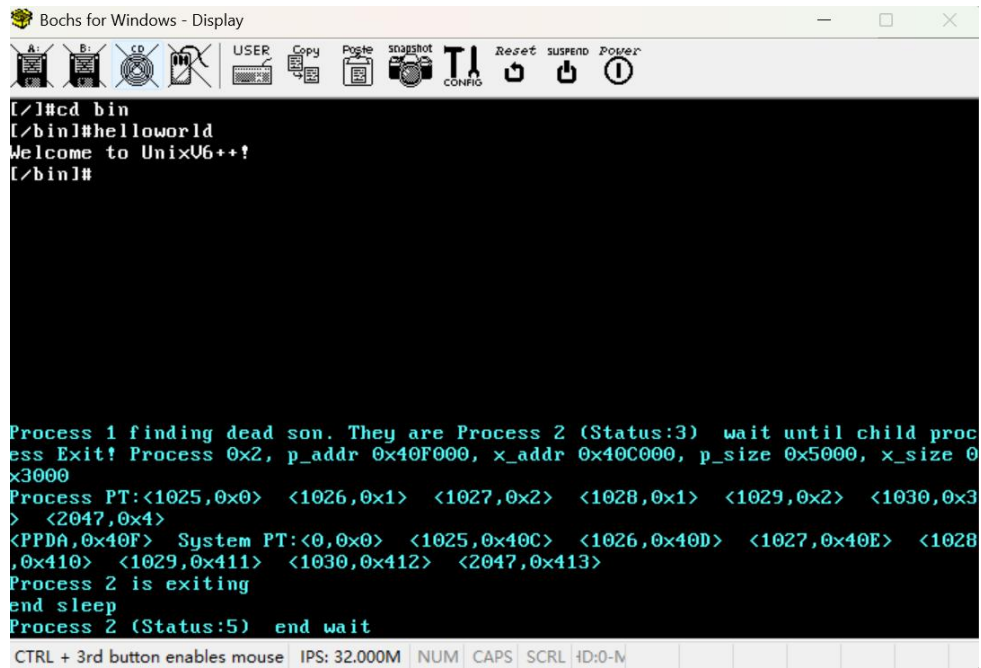
```
SHELL_OBJS =$(TARGET)\cat.exe \
            $(TARGET)\cp.exe \
            $(TARGET)\ls.exe \
            $(TARGET)\mkdir.exe \
            $(TARGET)\rm.exe \
            $(TARGET)\perf.exe \
            $(TARGET)\sig.exe \
            $(TARGET)\copyfile.exe \
            $(TARGET)\shutdown.exe \
            $(TARGET)\test.exe \
            $(TARGET)\forks.exe \
            $(TARGET)\trace.exe \
            $(TARGET)\echo.exe \
            $(TARGET)\date.exe \
            $(TARGET)\newsig.exe \
            $(TARGET)\sigTest.exe \
            $(TARGET)\stack.exe \
            $(TARGET)\malloc.exe \
            $(TARGET)\testSTDOUT.exe \
            $(TARGET)\fork.exe \
            $(TARGET)\showStack.exe \
            $(TARGET)\trivialProg.exe \
            $(TARGET)\helloworld.exe
```

3.添加 helloworld 的编译规则

```
$(TARGET)\helloworld.exe : helloworld.c
    $(CC) $(CFLAGS) -I"$(INCLUDE)" -I"$(LIB_INCLUDE)" $< -e _main1 $(V6++LIB) -o $@
    copy $(TARGET)\helloworld.exe $(MAKEIMAGEPATH)\$(BIN)\helloworld
```

4.执行 helloworld 程序

首先先在 OOS Command Prompt 中执行 all 命令重新制作系统盘，然后运行虚拟机，执行 helloworld 程序，成功实现功能。



```
[/]#cd bin
[/bin]#helloworld
Welcome to UnixV6++!
[/bin]#

Process 1 finding dead son. They are Process 2 (Status:3) wait until child proc
ess Exit! Process 0x2, p_addr 0x40F000, x_addr 0x40C000, p_size 0x5000, x_size 0
x3000
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x1> <1029,0x2> <1030,0x3
> <2047,0x4>
<PPDA,0x40F> System PT:<0,0x0> <1025,0x40C> <1026,0x40D> <1027,0x40E> <1028
,0x410> <1029,0x411> <1030,0x412> <2047,0x413>
Process 2 is exiting
end sleep
Process 2 (Status:5) end wait

CTRL + 3rd button enables mouse IPS: 32.000M NUM CAPS SCRL HD:0-M
```

part4: 用 Vscode 维护多套 Unix V6++ 系统

复制一份 oos, 命名为 oos1

MinGW	2024/9/10 1:56	文件夹
NASM	2024/9/10 1:56	文件夹
oos	2025/9/14 9:40	文件夹
oos1	2025/9/20 15:45	文件夹
添加新目录	2025/9/18 20:22	文件夹

oos_path 指向开发版的 oos 目录

```
@set oos_path=D:\desktop\OS\UNIX V6++V1\oos1
@set mingw_path=D:\desktop\OS\UNIX V6++V1\MinGW\bin
@set nasm_path=D:\desktop\OS\UNIX V6++V1\NASM
@set bochs_path=D:\desktop\OS\UNIX V6++V1\Bochs-2.6
@set BXSHARE=%bochs_path%
@set partcopy_path=%oos_path%\tools\partcopy

@set path=%partcopy_path%;%bochs_path%;%nasm_path%;%mingw_path%;%oos_path%;%path%

@cls
@echo Setting develop and build environment for UnixV6++.
```

首先尝试陈元睿学长的方法

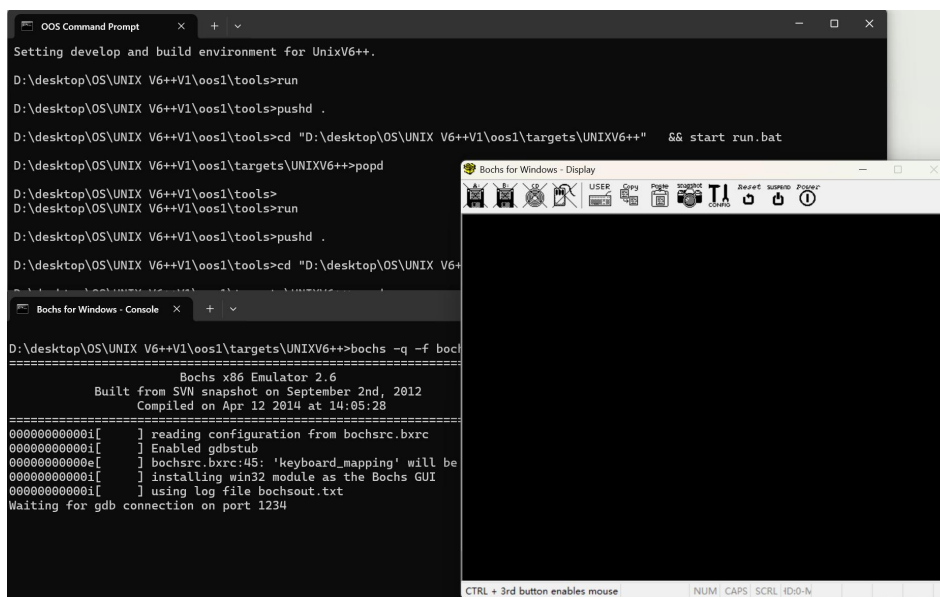
修改 launch.json

```

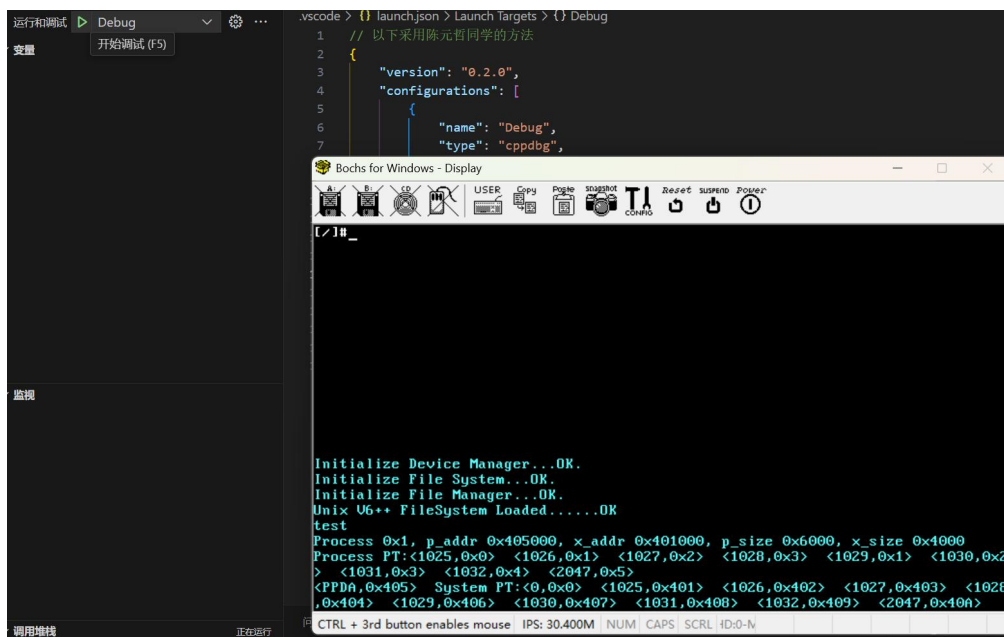
.vscode > {} launchjson > Launch Targets > {} Debug
1 // 以下采用陈元哲同学的方法
2 {
3     "version": "0.2.0",
4     "configurations": [
5     {
6         "name": "Debug",
7         "type": "cppdbg",
8         "request": "launch",
9         "miDebuggerServerAddress": "localhost:1234",
10        "program": "D:\\desktop\\OS\\UNIX V6++V1\\oos1\\targets\\objs\\kernel.exe",
11        "args": [],
12        "stopAtEntry": true,
13        "cwd": "${workspaceFolder}",
14        "externalConsole": false,
15        "MIMode": "gdb",
16        "preLaunchTask": "OOS run", // 前置任务: 启动Bochs
17    }
18 ]
19 }

```

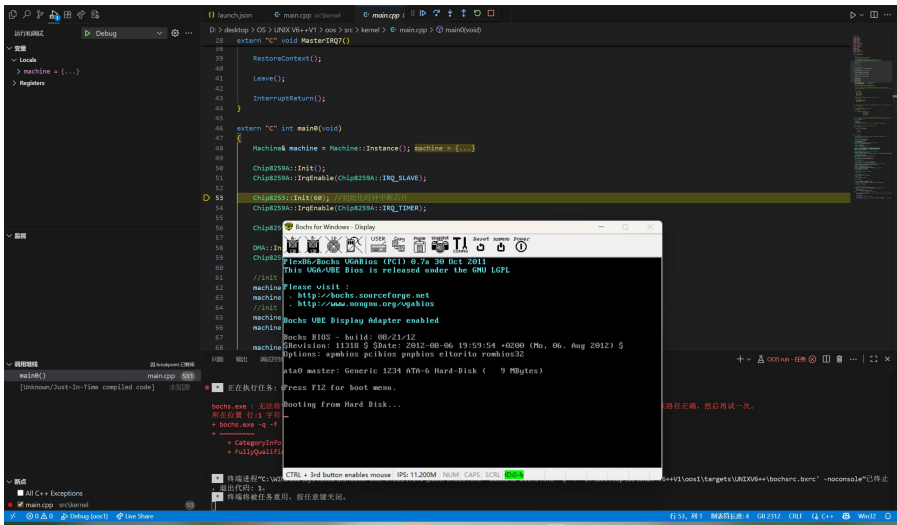
命令行界面输入 run 命令启动 Bochs 虚拟机



在 vscode 中启动调试

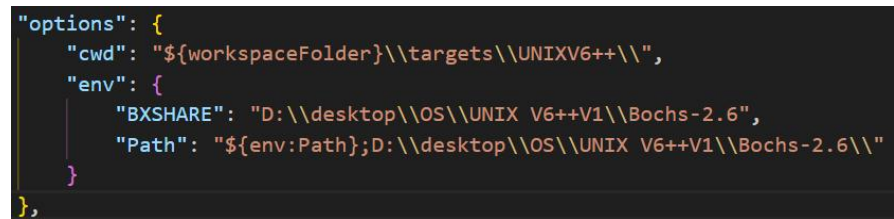


在 kernel 的 main 函数设置断点，开始调试，可以进行单步调试、逐过程等调试操作

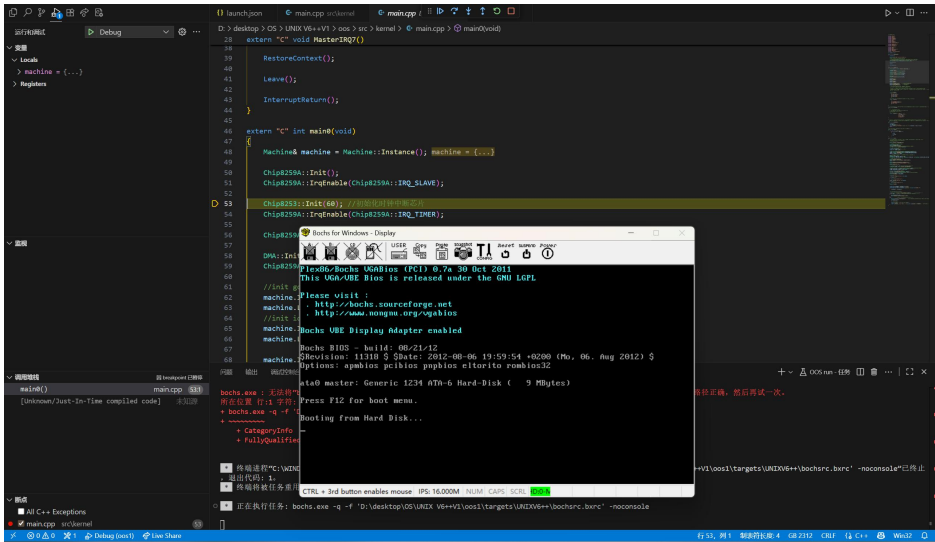


再尝试刘吉加学长的方式

修改 task.json



然后之间点击开始调试即可启动虚拟机，无需在命令行界面先启动



刘学长的方法相对于陈学长的改进在哪？

刘在 launch.json 中增加了 "preLaunchTask": "OOS run", 并在 tasks.json 中定义了 OOS run 任务。因此，当按下 F5 启动调试时，VS Code 首先执行 OOS run 任务来启动 Bochs，然后立刻无缝地将 GDB 调试器附加上去。所以不需要再在命令行界面先启动虚拟机了。